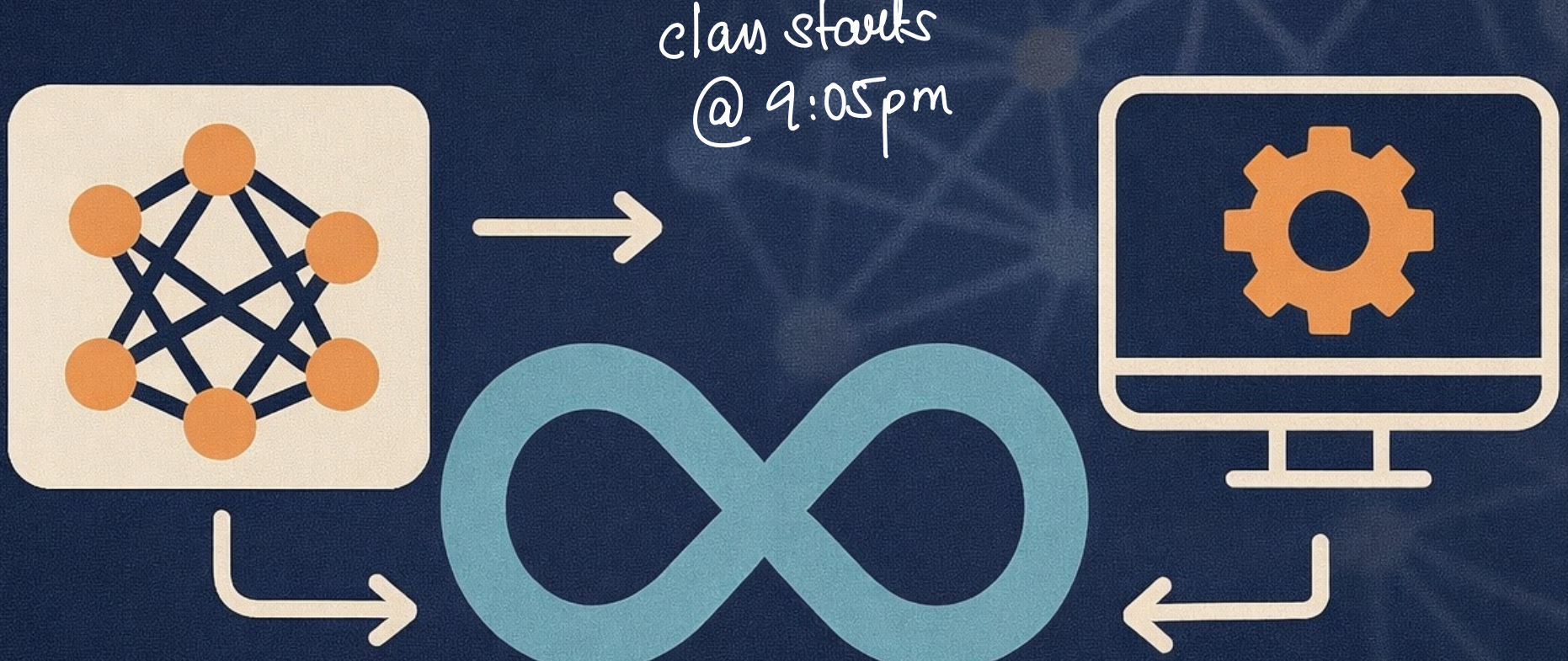
NEURAL NETWORK



Lecture : Neural Network Concepts

#Agenda

1) Dropout
2) Batch Normalization
3) Forward propagation
4) Backward propagation

} NN from scratch (using numpy)

Recap

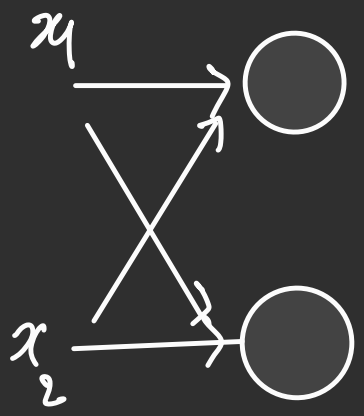
$R_1 \Rightarrow$ "I love pizza"

$\rightarrow WE_1 \rightarrow WE_2 \rightarrow WE_3 \longrightarrow [[WE_1], [WE_2], [WE_3]]$

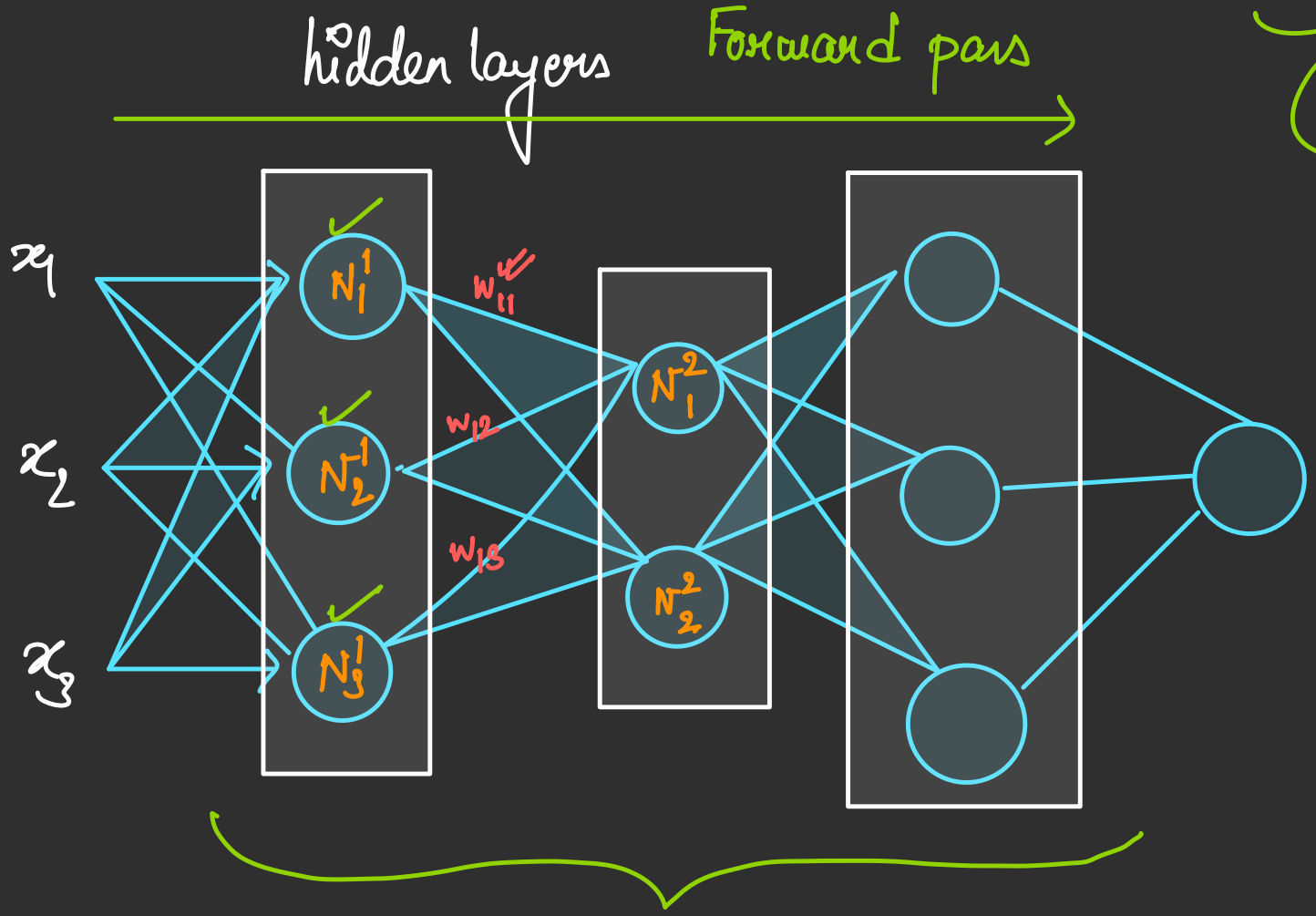
pooling $\rightarrow$ GAP, GMP

flatten $\rightarrow$ 1D

"No of parameters?"



Dropout



Overfitting

↳ great result on training data and bad result on test data

Epoch = No of Iteration

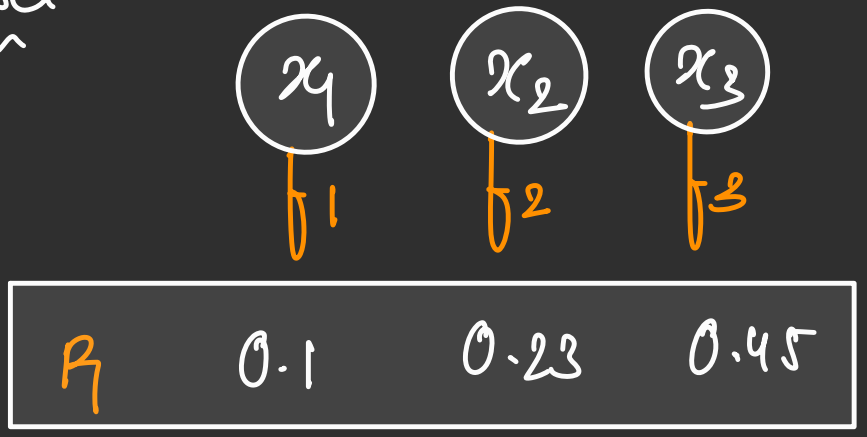
$E_1 \Rightarrow N_1' X$

$E_2 \Rightarrow N_3' X$

No of Nodes to turn off = HP

↳ % (50%)

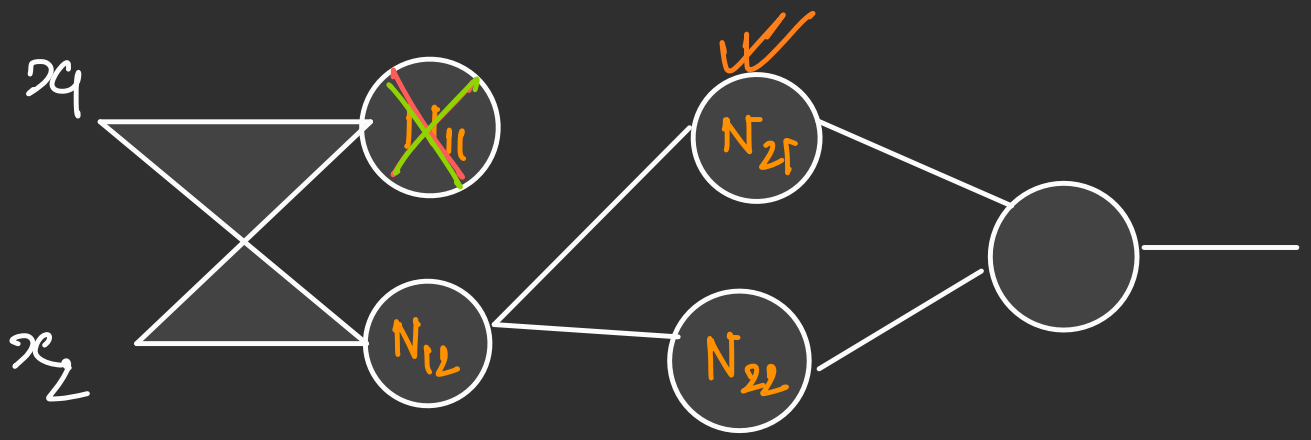
Dataset



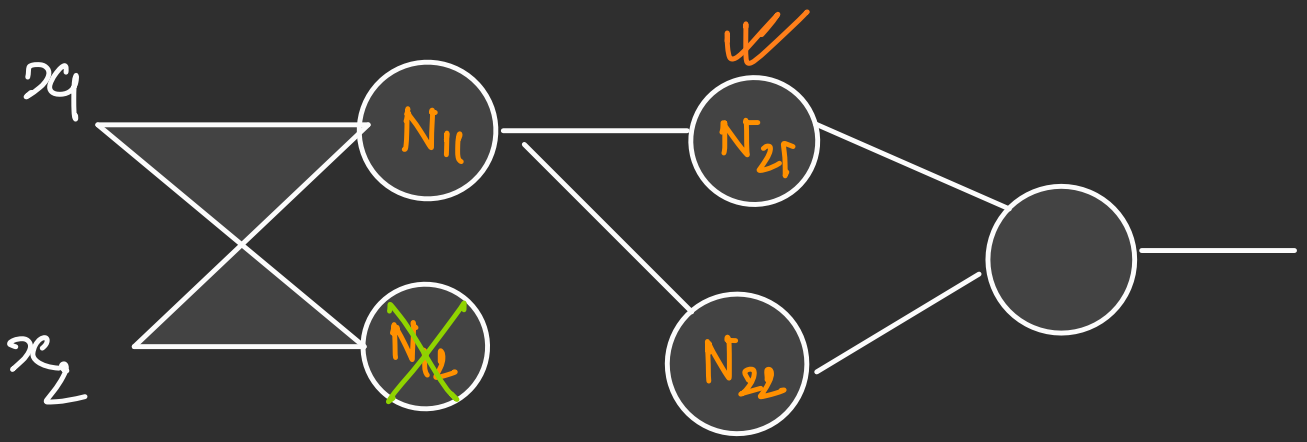
P_1	0.1	0.23	0.45
R_2	0.12	0.36	0.85
R_3	0.8	0.67	0.36

$$\begin{cases} x_1 = 0.1 \\ x_2 = 0.23 \\ x_3 = 0.45 \end{cases}$$

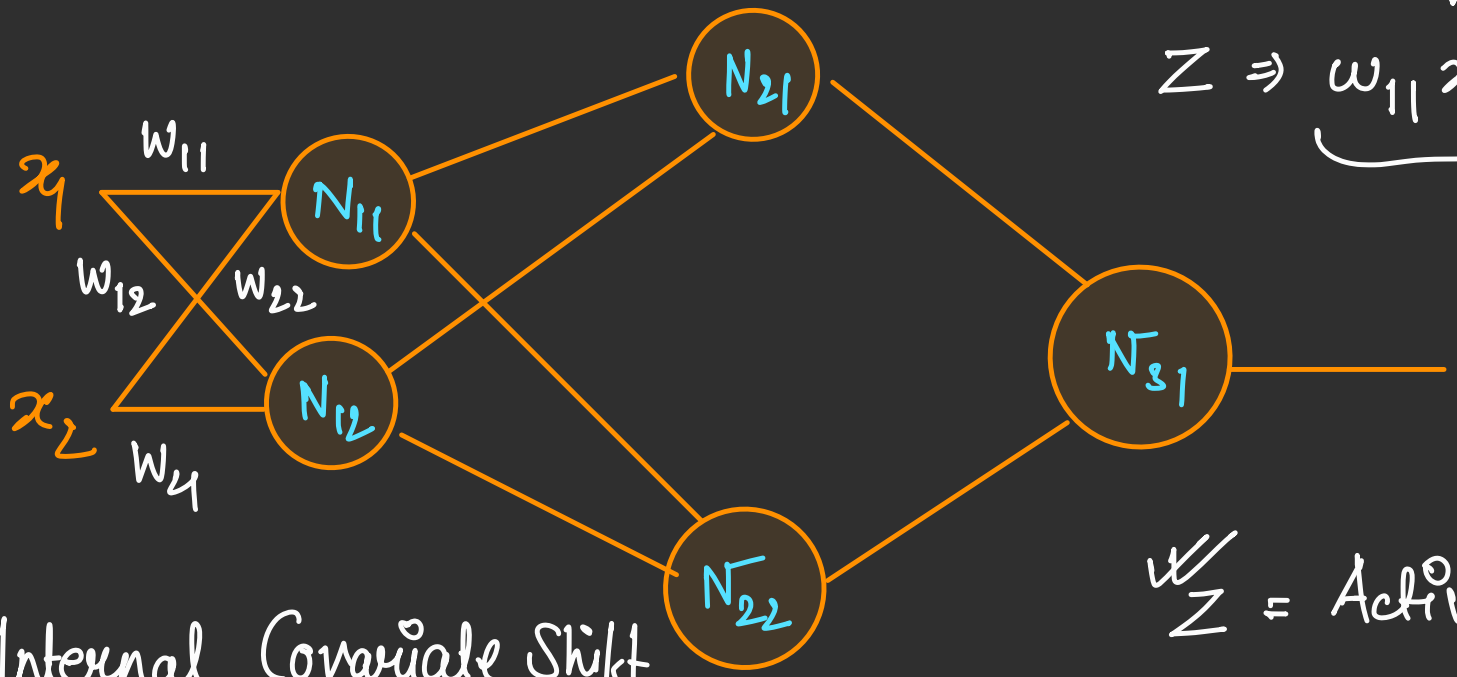
Epoch 1 =



Epoch 2 =



Batch Normalization

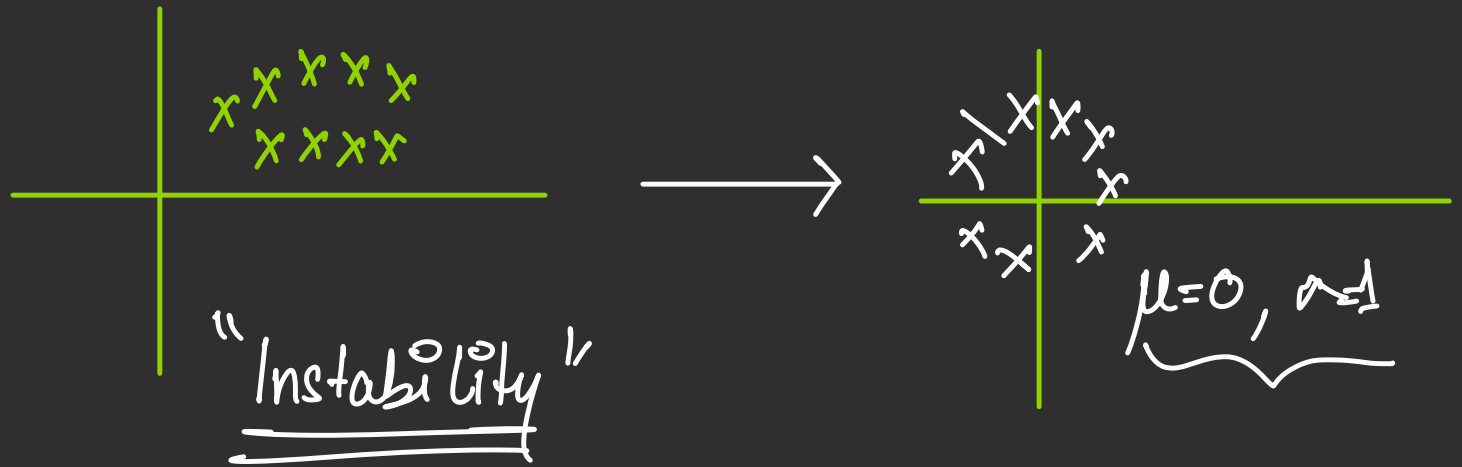


$N_{11} \Rightarrow \underbrace{w^T x + b}$

$Z \Rightarrow \underbrace{w_{11} x_1 + w_{12} x_2 + b}$

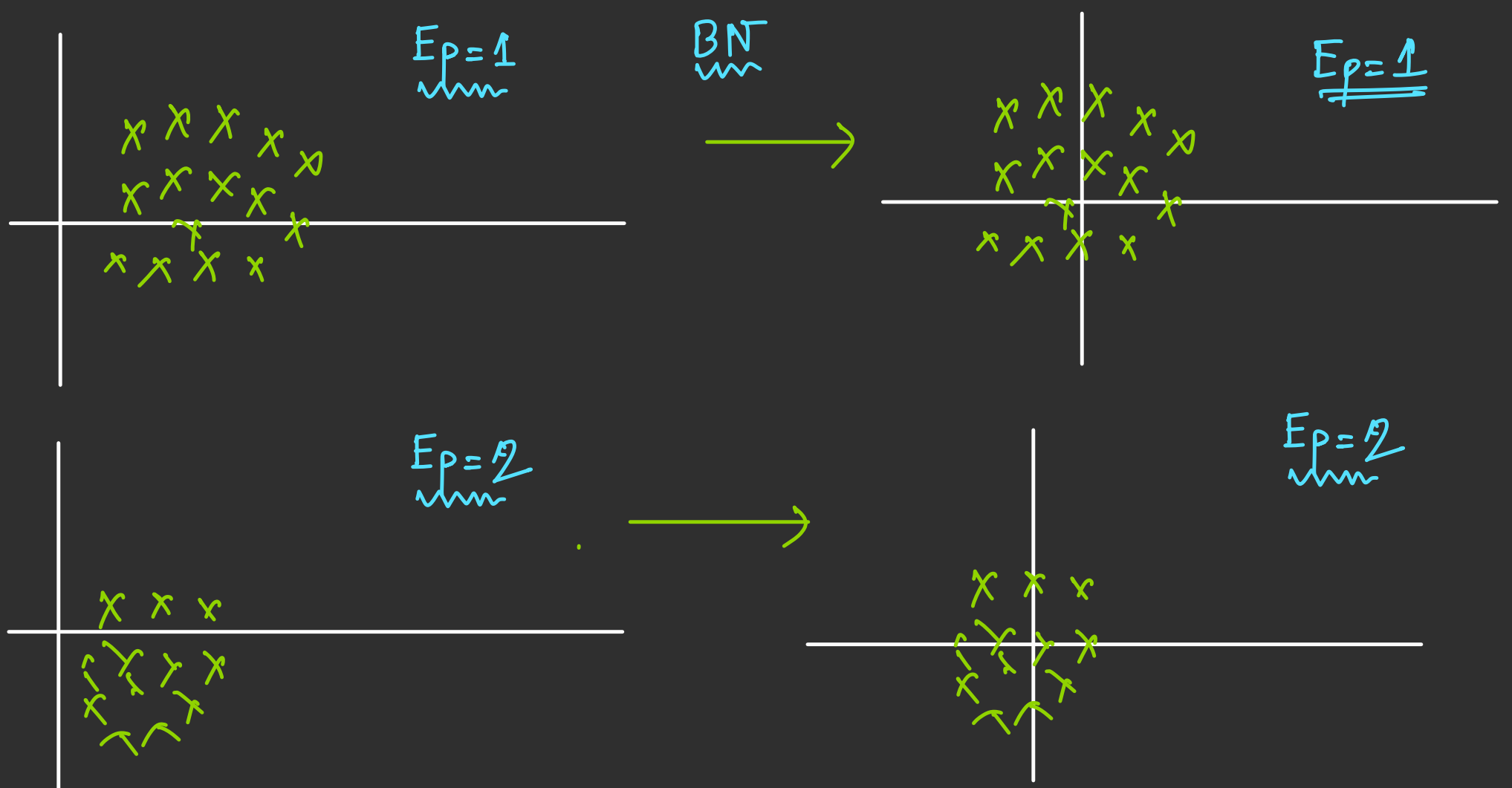
W_Z = Activation fun (Z)

Internal Covariate Shift



"Instability"

$\mu = 0, \sigma = 1$


$E_p = 1$
BN
$E_p = 1$
$E_p = 2$
$E_p = 2$

Batch Normalization

NN always take data in Batch

$R_1 \rightarrow R_{32}$

$\mu, \sigma \rightarrow \mu=0, \sigma=1$

B_1 Z_1 Z_2 Z_3 . . . .

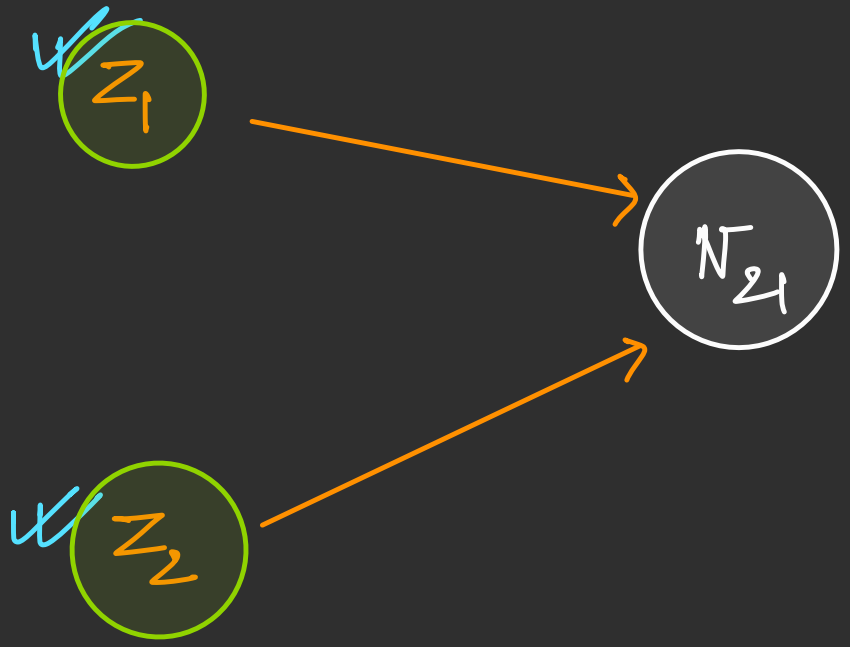
Same dist

B_2 Z_1 Z_2 Z_3 . . .

Same dist

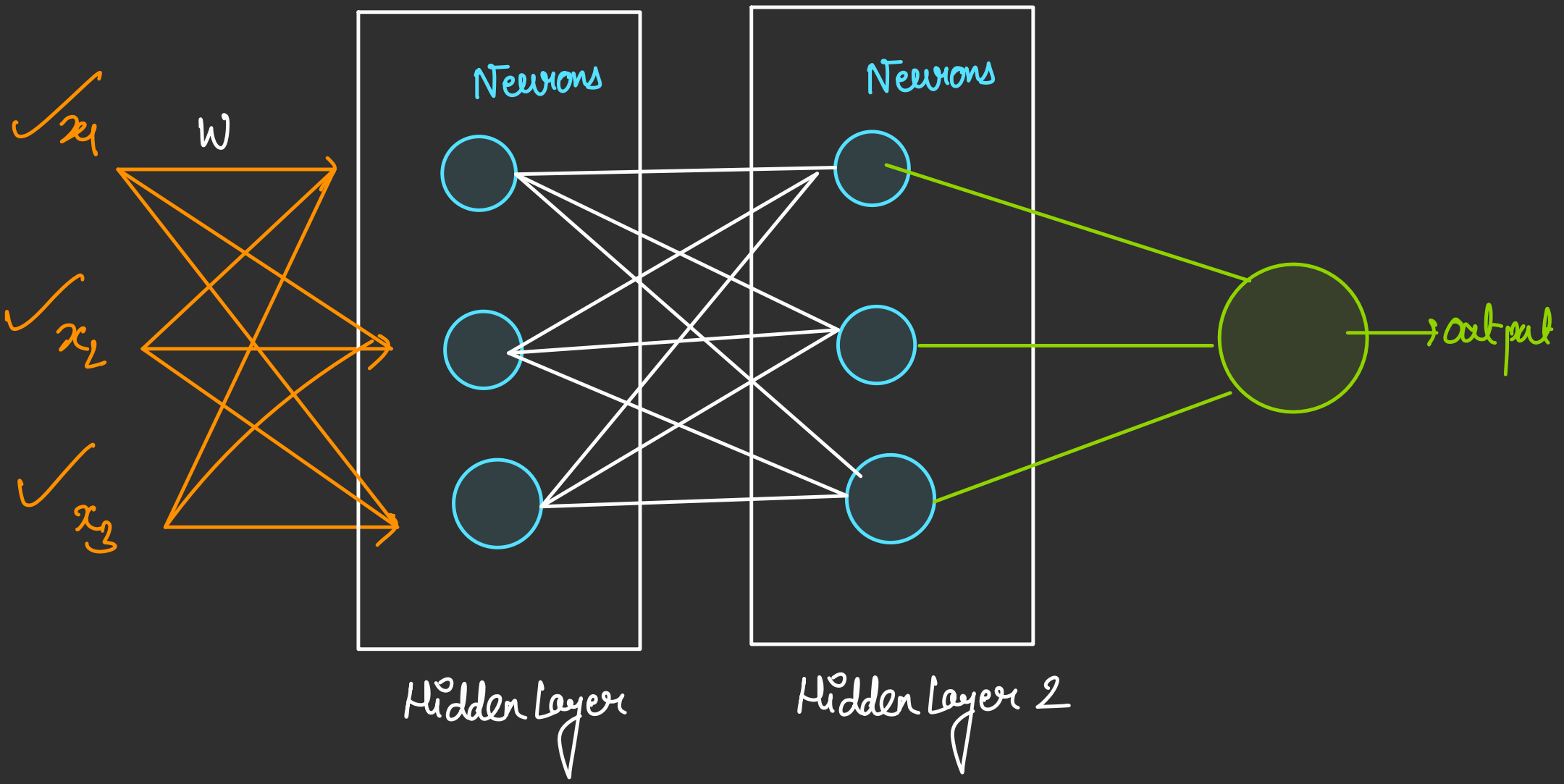
$$z_1 = w_{11} x_1 + w_{12} x_2 + b$$

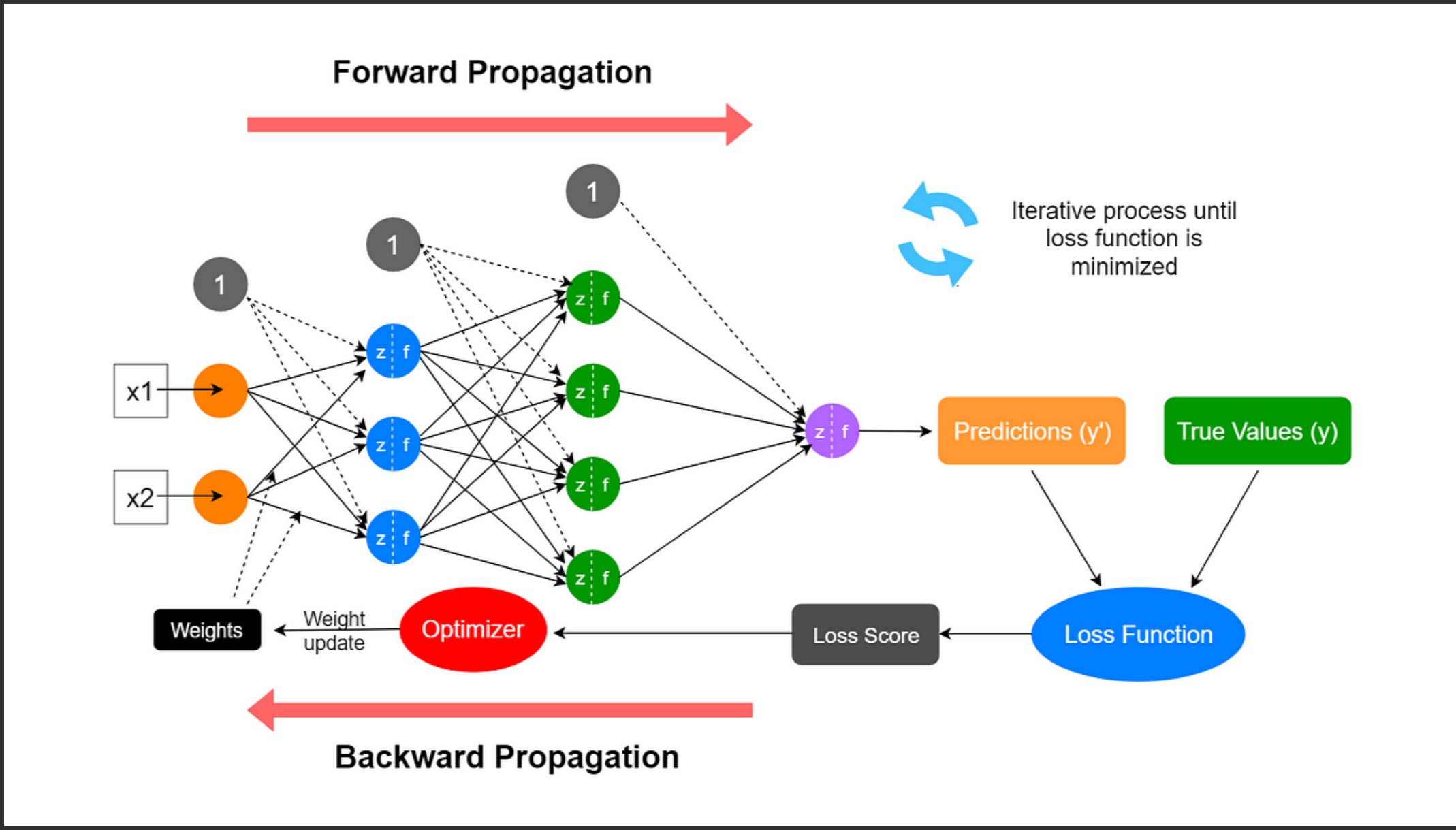
$$z_2 = w_{12} x_1 + w_{21} x_2 + b$$



(z_1, z_2)

Neural Network (FC layers)




Forward Propagation
1
1
1
x1
x2
z f
z f
z f
z f
z f
z f
z f
Iterative process until loss function is minimized
Predictions (y')
True Values (y)
Loss Function
Loss Score
Optimizer
Weight update
Weights
Backward Propagation

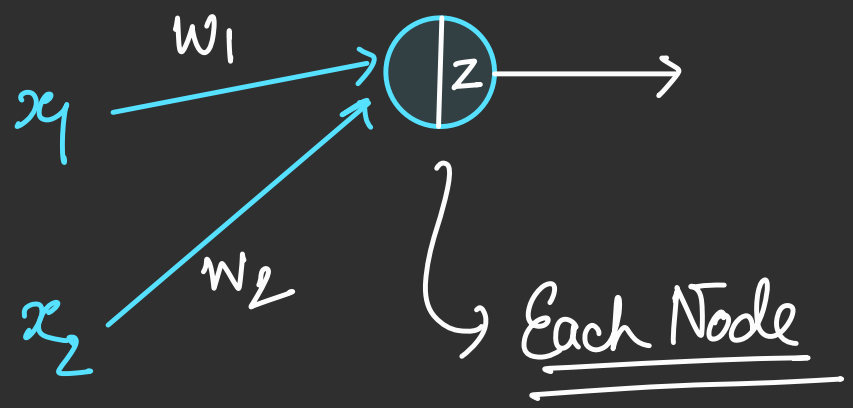
Forward prop

$$z = w_1 x_1 + w_2 x_2 + b$$

$$Z = \text{activation fun}^n(z)$$

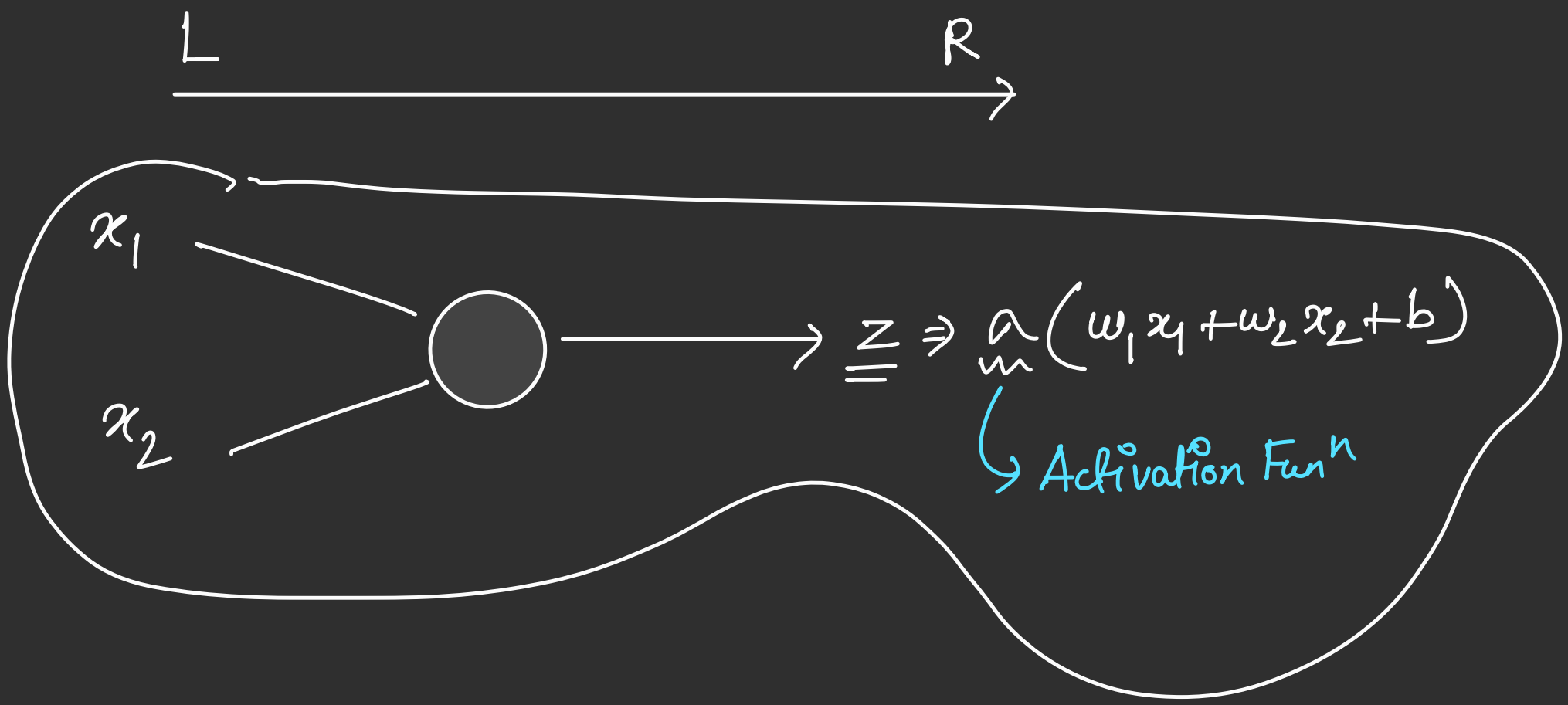
relu, sigmoid

tanh, leaky relu





Activation Fun^{n} $\longrightarrow$ make DB Non-linear

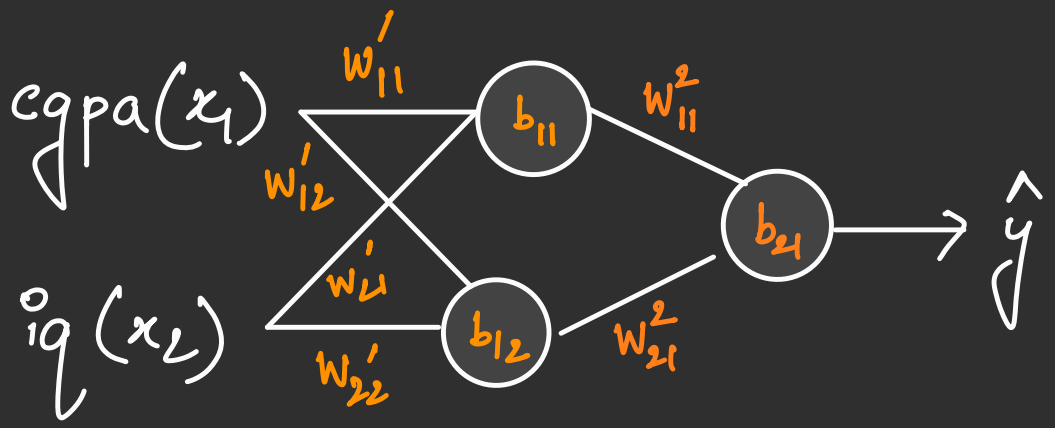


$$\begin{cases} w_1, w_2 \longrightarrow 1 \\ b = 0 \end{cases}$$

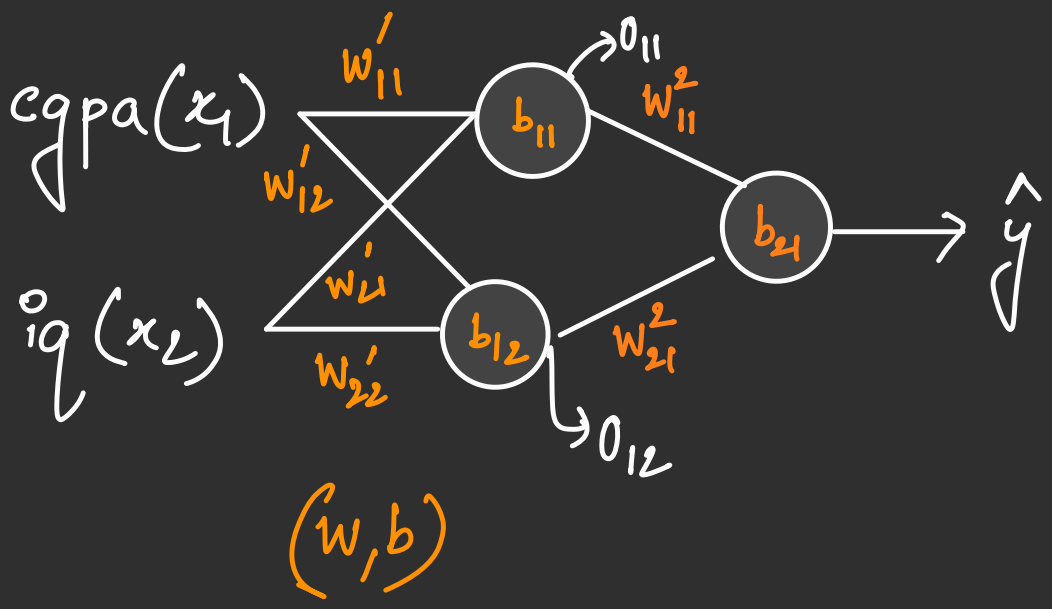
Back propagation

↳ Algo that helps to train NN

cgpa ✓	iq ✓	4pa ✗
8	80	8
7	70	7



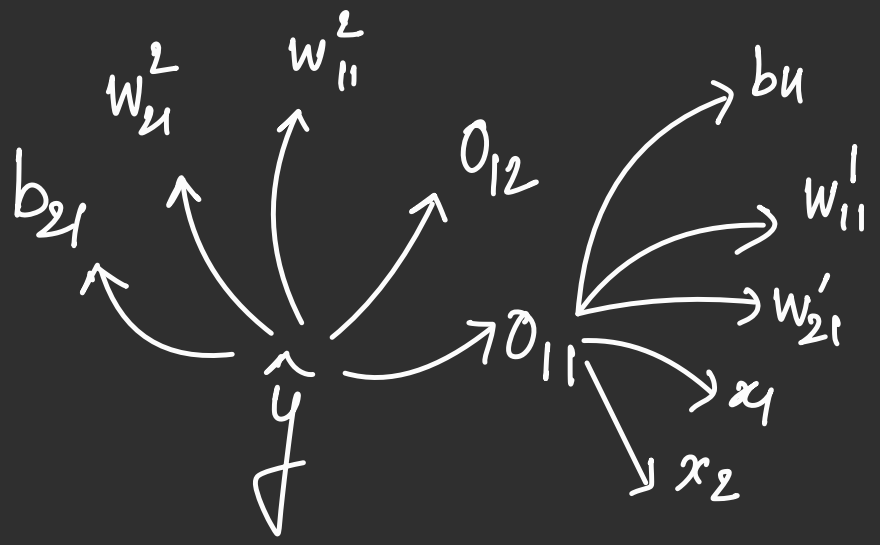
(W, b)



Steps

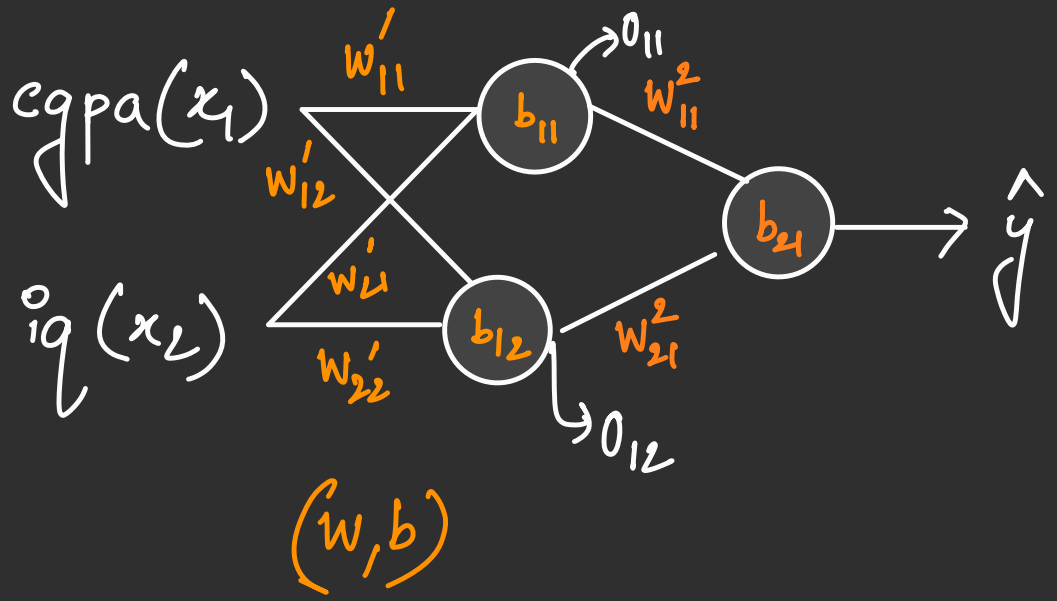
1) Take one row (data point)
2) predict/get $\hat{y}$ → Forward prop
3) Choose a loss function (MSE)
4) Weights & bias → GD

$$L = (y - \hat{y})^2$$



$$W_{new} = W_{old} - \eta \frac{\partial L}{\partial W_{old}}$$

$$B_{new} = B_{old} - \eta \frac{\partial L}{\partial B_{old}}$$

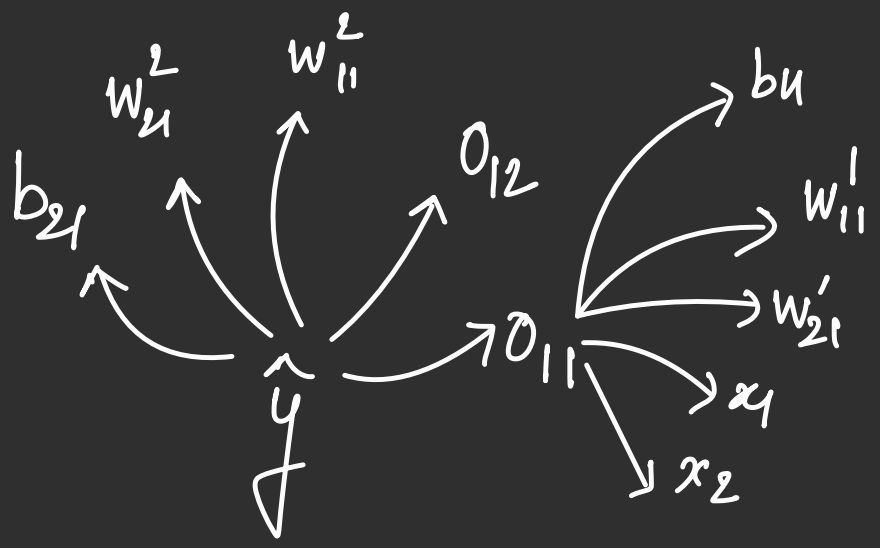


$$w^2_{11_{new}} = w^2_{11_{old}} - \eta \frac{\partial L}{\partial w^2_{11_{old}}}$$

wq Such Eqn

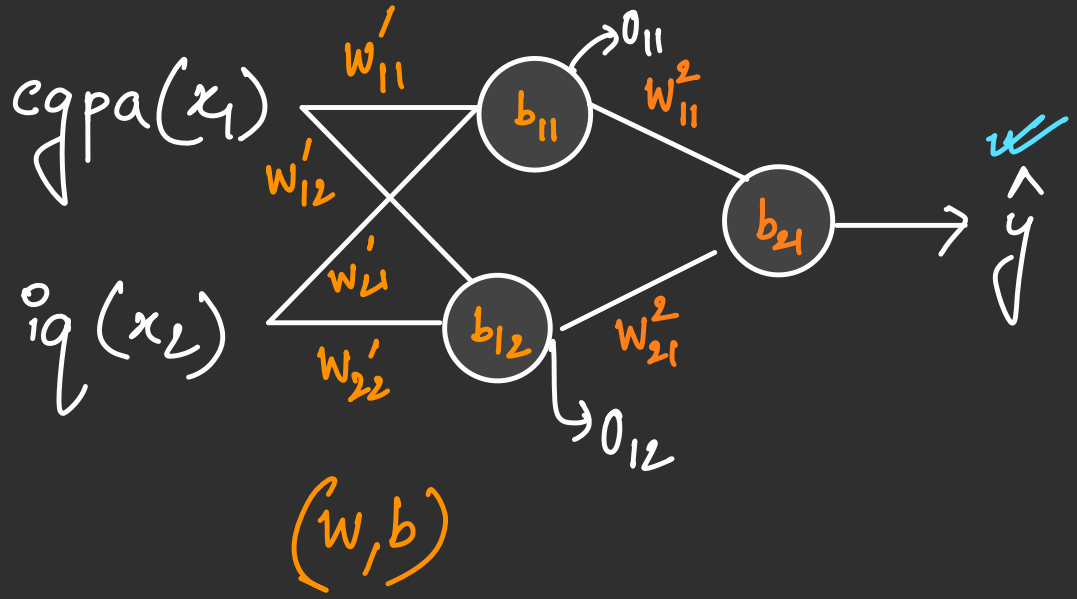
$$\frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w^2_{11}}$$

Chain Rule



$$W_{new} = W_{old} - \eta \frac{\partial L}{\partial W_{old}}$$

$$B_{new} = B_{old} - \eta \frac{\partial L}{\partial B_{old}}$$



$$w^2_{11_{new}} = w^2_{11_{old}} - \eta \frac{\partial L}{\partial w^2_{11_{old}}}$$

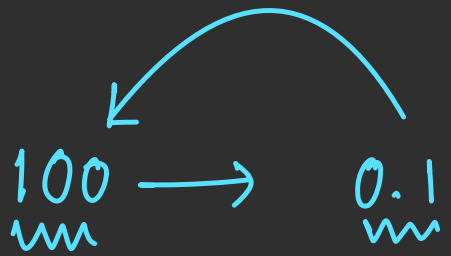
w/q Such Eq^n

$$\frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w^2_{11}}$$

Chain Rule

$$\frac{\partial L}{\partial w^2_{11}} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w^2_{11}}$$

$$\frac{\partial L}{\partial w^1_{11}} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w^2_{11}} \cdot \frac{\partial w^2_{11}}{\partial w_{11}}$$


100
0.1

Doubts

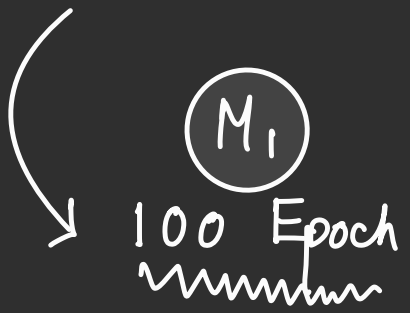
1) NN → without Dropout

Dropout → reduce overfitting

Underfitting → 1) Introduce more data

2) Make arch complex by adding more hidden layer

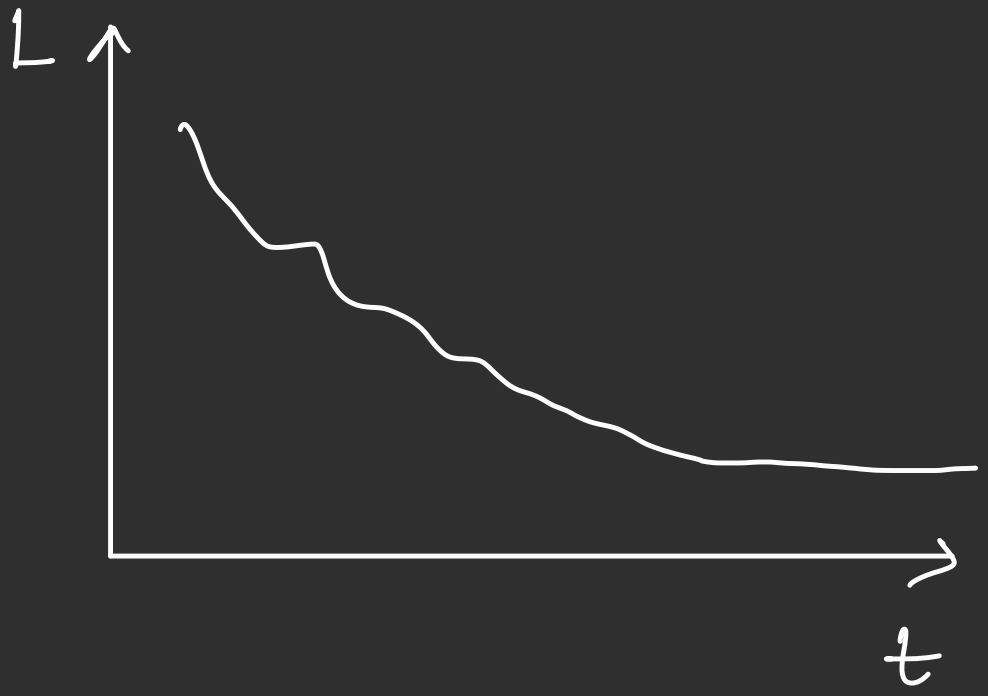
2) More epoch



$L_{82} = 0.001$

$L_{83} = 0.001$

$L_{84} = 0.001$



3)

Epoch

↳ architecture level

(Epoch = 15)

Batch Norm / Drop out

↳ Layers level

4) Activation Fun^n

↳ Help us/NN learn non-linear decision boundary

Without Activation Function

Layer 1: $y_1 = w_1 x + b_1$

Layer 2: $y_2 = w_2 y_1 + b_2$

Final: $y_2 = w_2(w_1 x + b_1) + b_2$

$\Rightarrow w_2 w_1 x_1 + w_2 b_1 + b_2$

Layer 1 = Relu $(w_1 x + b_1) \Rightarrow$

Layer 2 $\Rightarrow y_2 = w_2 (\text{Relu}(w_1 x + b_1)) + b_2$

↳ Non-linear

problems

1) Hard to tune
2) Data Hungry
3) Interpretibity

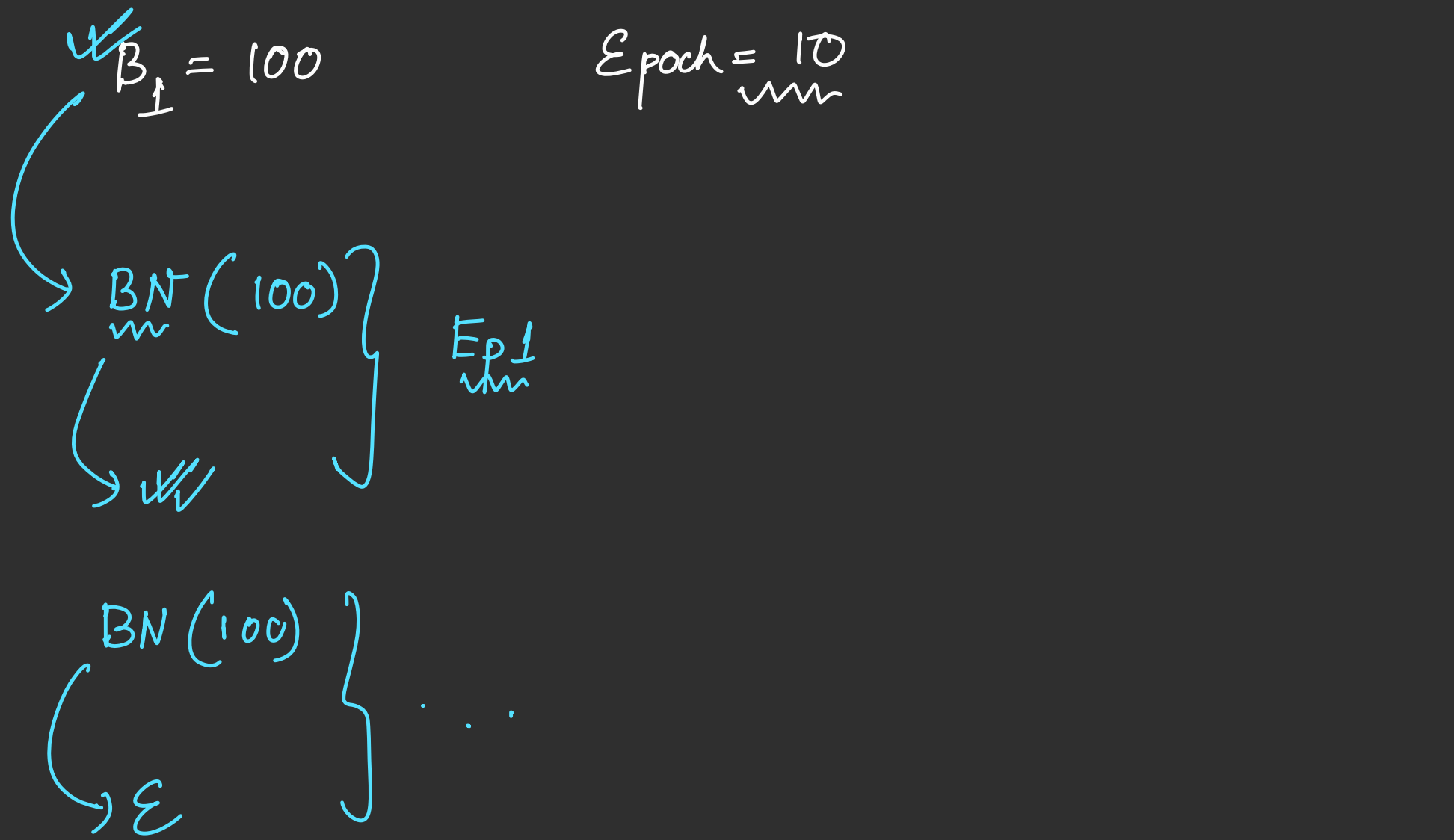
$\mathcal{E}_{bank} = 15$

$B_1 = 10$, $B_2 = 10$, $B_3 = 10$

B_1 → $15\mathcal{E}$

B_2 → $15\mathcal{E}$

B_3 → $15\mathcal{E}$


B1 = 100
Epoch = 10
BN (100)
Ep 1
BN (100)
E
. . .